

Theme: Medical ICU Bedside ECG Telemetry & Nurse Call System – Real- Time Systems

Final Capstone

A dual-core FreeRTOS bedside monitor that simulates ECG/HR/O2 telemetry through a producer, consumer, coordinator, responder chain built to demonstrate the detection of arrhythmias and hypoxemia in real-time with hardware nurse-call override and demonstrate the deterministic embedded IPC design for an embedded/firmware engineering role in relation to medical device software.

Demo:

- **Video Link:** <https://youtu.be/sQ4TaH5XAuQ>
- **Live Wokwi:** <https://wokwi.com/projects/470915869059743745>

Architecture:

- The way that this application works is that sensor data is sampled every 50ms by task A and placed into a FreeRTOS queue. Then it's consumed by task B which then performs the arrhythmia detection and then updates the shared patient state. After each is produced and the processing is complete, then the event group synchronizes task C which notifies task D to dispatch the alarm or the nurse-call responses. All the health metrics including the WCET, queue depth, and fault counters are then exported to the core 0 monitoring

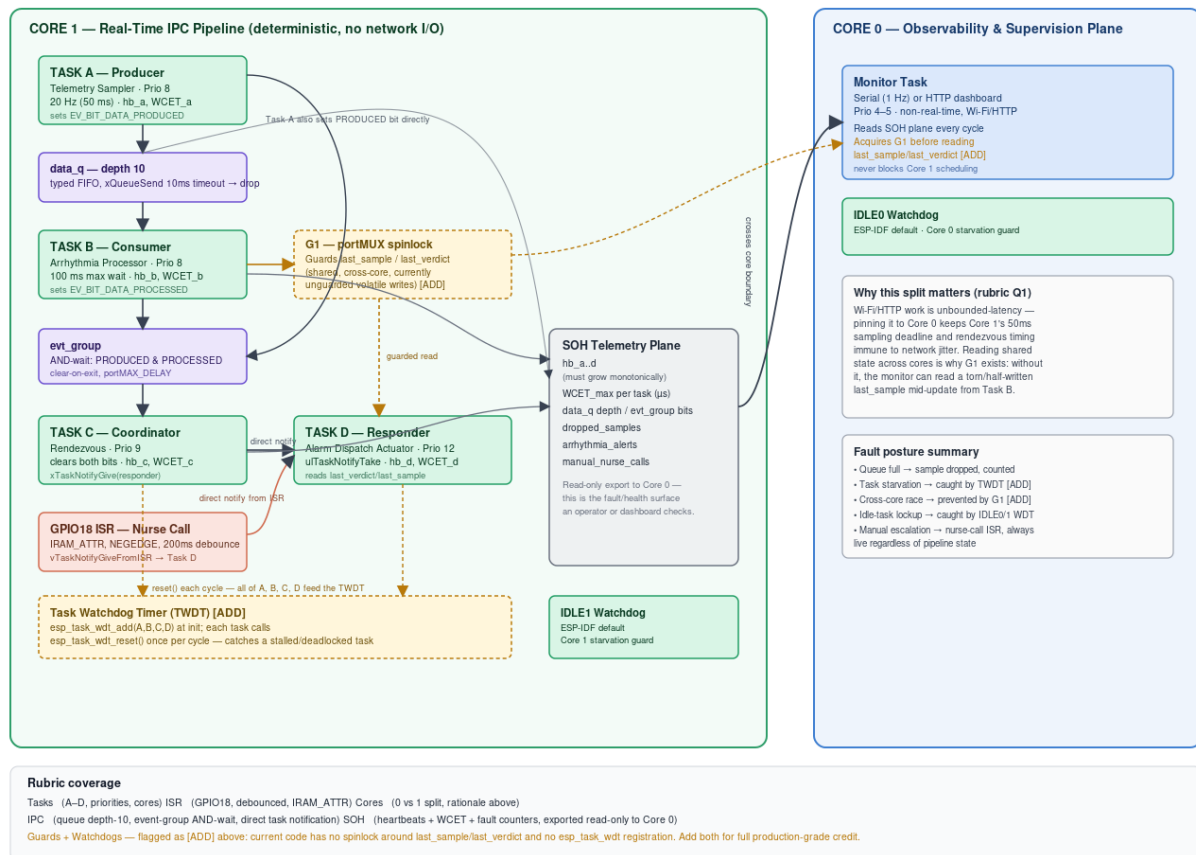
task while the real-time pipeline is still isolated on core 1 in order to maintain the deterministic execution.

- **Figure 1:** Final Integration Capstone Architecture that shows the complete dual-core implementation, ISR integration, IPC mechanisms, watchdogs, and system health monitoring.

Final Integration Capstone — Production-Grade Concurrency Architecture

ICU Bedside ECG Telemetry & Nurse Call System — tasks · ISR · cores · IPC · guards · watchdogs · SOH

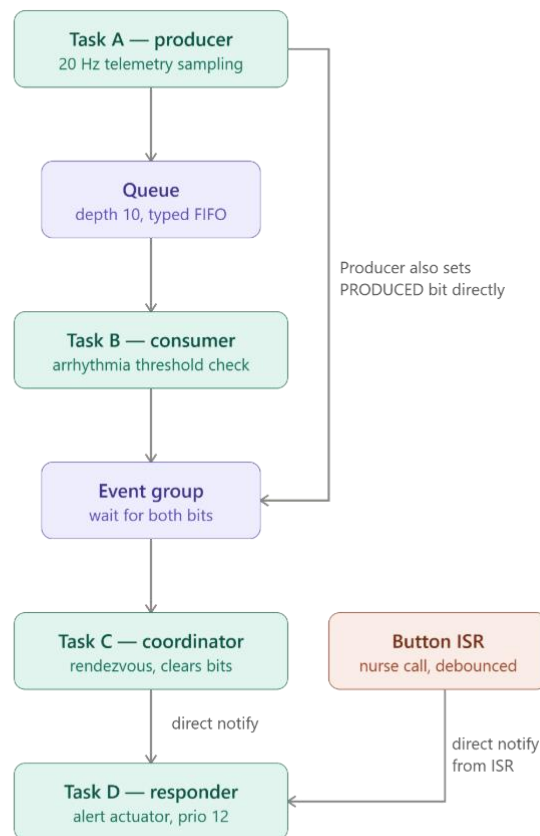
Implemented in current firmware
Recommended addition (guard / watchdog)



Concurrency Diagrams:

- **The Data/Control Flow:** All the teal boxes are the four core-1 tasks and purple boxes are the two shared IPC objects while coral is the external interrupt source.

- **Figures 2 & 3:** The original concurrency diagrams from application 5 that show the producer/consumer pipeline, synchronization primitives, and task relationships.



- **The core-placement view:**



Tasks & timing (WCET Evidence):

Task table & WCET utilization calculator

Enter each periodic task with its period T and measured worst-case execution time C (same time unit). This computes per-task utilization $U_i = C/T$, total utilization, and checks it against the schedulability bounds. Export the finished table into your portfolio as your WCET evidence.

TASK	PERIOD T	WCET C	$U_i = C/T$	DEL
A-> Telemetry Sampler	50ms	0.030ms	0.001	×
B-> Arrhythmia Processor	50ms	0.164ms	0.003	×
C-> Pipeline Coordinator	50ms	0.079ms	0.002	×
D-> Alarm Dispatch Actuator	50ms	0.008ms	0.000	×

+ Add task

Reset example

0.01

TOTAL UTILIZATION $U = \sum C_i/T_i$

✓ $U = 0.006 \leq \text{RM bound (0.757)}$. Schedulable under Rate-Monotonic and EDF (independent periodic tasks, deadline = period).

Task	Period T	WCET C	U = C/T	Priority	Deadline
Task A -> Telemetry Sampler	50 ms	0.030 ms	0.0006	8	50 ms
Task B -> Arrhythmia Processor	50 ms	0.164 ms	0.00328	8	50 ms
Task C -> Pipeline Coordinator	50 ms	0.079 ms	0.00158	9	50 ms
Task D -> Alarm Dispatch Actuator	50 ms	0.008 ms	0.00016	12	50 ms

- Total Utilization $U = 0.00060 + 0.00328 + 0.00158 + 0.00016 = 0.00562$
- Rate Monotonic: $U = 0.00562 < 0.757 \rightarrow$ schedulable!
- Earliest Deadline: Since $U < 1.0$ then the task set is schedulable under EDF.

Task Table & WCET Utilization Calculator Values from Demo Run:

== ICU ECG Telemetry Monitor · Dual-Core IPC Pipeline ==

Telemetry: Status=NORMAL | HR=60 BPM | SpO2=99% | ECG=-100 mV

IPC State: Queue Depth=0/10 | EventBits=0x00

Task	Period	Priority	Heartbeats	WCET(us)
------	--------	----------	------------	----------

A: Telemetry Sampler (Prod)	50 ms	8	1	19
-----------------------------	-------	---	---	----

B: Arrhythmia Process (Cons)	Queue	8	1	4
------------------------------	-------	---	---	---

C: Pipeline Coordinator	Rendezvous	9	1	
-------------------------	------------	---	---	--

D: Alarm Dispatch Actuator	Notif	12	0	0
----------------------------	-------	----	---	---

Stats: Auto Alerts=0 | Nurse Calls=0 | Drops=0

=== ICU ECG Telemetry Monitor · Dual-Core IPC Pipeline ===

Telemetry: Status=NORMAL | HR=79 BPM | SpO2=98% | ECG=90 mV

IPC State: Queue Depth=0/10 | EventBits=0x00

Task	Period	Priority	Heartbeats	WCET(us)
A: Telemetry Sampler (Prod)	50 ms	8	21	30
B: Arrhythmia Process (Cons)	Queue	8	21	164
C: Pipeline Coordinator	Rendezvous	9	21	79
D: Alarm Dispatch Actuator	Notif	12	20	8

Stats: Auto Alerts=3 | Nurse Calls=0 | Drops=0

=== ICU ECG Telemetry Monitor · Dual-Core IPC Pipeline ===

Telemetry: Status=NORMAL | HR=99 BPM | SpO2=96% | ECG=90 mV

IPC State: Queue Depth=0/10 | EventBits=0x00

Task	Period	Priority	Heartbeats	WCET(us)
A: Telemetry Sampler (Prod)	50 ms	8	41	30
B: Arrhythmia Process (Cons)	Queue	8	41	164

C: Pipeline Coordinator	Rendezvous	9	41	79
-------------------------	------------	---	----	----

D: Alarm Dispatch Actuator	Notif	12	41	8
----------------------------	-------	----	----	---

Stats: Auto Alerts=6 | Nurse Calls=1 | Drops=0

=== ICU ECG Telemetry Monitor · Dual-Core IPC Pipeline ===

Telemetry: Status=ARRHYTHMIA | HR=64 BPM | SpO2=94% | ECG=90 mV

IPC State: Queue Depth=0/10 | EventBits=0x00

Task	Period	Priority	Heartbeats	WCET(us)
------	--------	----------	------------	----------

A: Telemetry Sampler (Prod)	50 ms	8	61	30
-----------------------------	-------	---	----	----

B: Arrhythmia Process (Cons)	Queue	8	61	164
------------------------------	-------	---	----	-----

C: Pipeline Coordinator	Rendezvous	9	61	79
-------------------------	------------	---	----	----

D: Alarm Dispatch Actuator	Notif	12	62	8
----------------------------	-------	----	----	---

Stats: Auto Alerts=21 | Nurse Calls=2 | Drops=0

=== ICU ECG Telemetry Monitor · Dual-Core IPC Pipeline ===

Telemetry: Status=NORMAL | HR=84 BPM | SpO2=98% | ECG=90 mV

IPC State: Queue Depth=0/10 | EventBits=0x00

Task	Period	Priority	Heartbeats	WCET(us)
------	--------	----------	------------	----------

A: Telemetry Sampler (Prod)	50 ms	8	81	30
-----------------------------	-------	---	----	----

B: Arrhythmia Process (Cons)	Queue	8	81	164
------------------------------	-------	---	----	-----

C: Pipeline Coordinator	Rendezvous	9	81	79
-------------------------	------------	---	----	----

D: Alarm Dispatch Actuator	Notif	12	83	8
----------------------------	-------	----	----	---

Stats: Auto Alerts=24 | Nurse Calls=3 | Drops=0

=== ICU ECG Telemetry Monitor · Dual-Core IPC Pipeline ===

Telemetry: Status=ARRHYTHMIA | HR=104 BPM | SpO2=96% | ECG=90 mV

IPC State: Queue Depth=0/10 | EventBits=0x00

Task	Period	Priority	Heartbeats	WCET(us)
------	--------	----------	------------	----------

A: Telemetry Sampler (Prod)	50 ms	8	101	30
-----------------------------	-------	---	-----	----

B: Arrhythmia Process (Cons)	Queue	8	101	164
------------------------------	-------	---	-----	-----

C: Pipeline Coordinator	Rendezvous	9	101	79
-------------------------	------------	---	-----	----

D: Alarm Dispatch Actuator	Notif	12	103	8
----------------------------	-------	----	-----	---

Stats: Auto Alerts=32 | Nurse Calls=3 | Drops=0

=== ICU ECG Telemetry Monitor · Dual-Core IPC Pipeline ===

Telemetry: Status=ARRHYTHMIA | HR=69 BPM | SpO2=94% | ECG=90 mV

IPC State: Queue Depth=0/10 | EventBits=0x00

Task	Period	Priority	Heartbeats	WCET(us)
A: Telemetry Sampler (Prod)	50 ms	8	121	30
B: Arrhythmia Process (Cons)	Queue	8	121	164
C: Pipeline Coordinator	Rendezvous	9	121	79
D: Alarm Dispatch Actuator	Notif	12	124	8

Stats: Auto Alerts=43 | Nurse Calls=4 | Drops=0

=== ICU ECG Telemetry Monitor · Dual-Core IPC Pipeline ===

Telemetry: Status=NORMAL | HR=89 BPM | SpO2=98% | ECG=90 mV

IPC State: Queue Depth=0/10 | EventBits=0x00

Task	Period	Priority	Heartbeats	WCET(us)
A: Telemetry Sampler (Prod)	50 ms	8	141	30
B: Arrhythmia Process (Cons)	Queue	8	141	164
C: Pipeline Coordinator	Rendezvous	9	141	79
D: Alarm Dispatch Actuator	Notif	12	144	8

Stats: Auto Alerts=46 | Nurse Calls=4 | Drops=0

=== ICU ECG Telemetry Monitor · Dual-Core IPC Pipeline ===

Telemetry: Status=ARRHYTHMIA | HR=109 BPM | SpO2=96% | ECG=90 mV

IPC State: Queue Depth=0/10 | EventBits=0x00

Task	Period	Priority	Heartbeats	WCET(us)
A: Telemetry Sampler (Prod)	50 ms	8	161	30
B: Arrhythmia Process (Cons)	Queue	8	161	164
C: Pipeline Coordinator	Rendezvous	9	161	79
D: Alarm Dispatch Actuator	Notif	12	165	8

Stats: Auto Alerts=58 | Nurse Calls=5 | Drops=0

=== ICU ECG Telemetry Monitor · Dual-Core IPC Pipeline ===

Telemetry: Status=ARRHYTHMIA | HR=74 BPM | SpO2=94% | ECG=90 mV

IPC State: Queue Depth=0/10 | EventBits=0x00

Task	Period	Priority	Heartbeats	WCET(us)
A: Telemetry Sampler (Prod)	50 ms	8	181	30
B: Arrhythmia Process (Cons)	Queue	8	181	164
C: Pipeline Coordinator	Rendezvous	9	181	79
D: Alarm Dispatch Actuator	Notif	12	185	8

Stats: Auto Alerts=65 | Nurse Calls=5 | Drops=0

=== ICU ECG Telemetry Monitor · Dual-Core IPC Pipeline ===

Telemetry: Status=NORMAL | HR=94 BPM | SpO2=98% | ECG=90 mV

IPC State: Queue Depth=0/10 | EventBits=0x00

Task	Period	Priority	Heartbeats	WCET(us)
A: Telemetry Sampler (Prod)	50 ms	8	201	30
B: Arrhythmia Process (Cons)	Queue	8	201	164
C: Pipeline Coordinator	Rendezvous	9	201	79
D: Alarm Dispatch Actuator	Notif	12	205	8

Stats: Auto Alerts=68 | Nurse Calls=5 | Drops=0

Induced-failure Prediction vs. Observation Analysis:

I decided to test what rapidly pressing the button does to the outputs to the terminal. I predicted initially that the GPIO ISR would remain responsive since it should perform minimal work and it should also wake up task D whiles using direct task notification. Whiles I tested it, I noticed that the nurse-call counter increased every time I pressed the button as it should, and I noticed that the telemetry pipeline continued to properly update normally. All other measurements and readings that were displayed to the terminal normally as well. This helped me to conclude that my initial hypothesis was right, and the GPIO ISR does remain responsive.

Hazard Analysis & Standard Mapping:

Hazard	Potential Effect	Mitigation Implemented	Standard Mapping
Telemetry queue overflow	Patient telemetry samples may be dropped -> delays the arrhythmia detection	Queue depth of 10 provides a 500 ms buffer. Queue failures increment a dropped-sample counter and are monitored continuously, so overload is visible during testing.	IE IEC 62304 §5.5 – Software implementation and defensive error handling
Missed arrhythmia detection due to delayed processing	Critical cardiac events may not trigger an alarm in time.	Producer, consumer, coordinator, and responder execute deterministic FreeRTOS tasks with measured WCET, fixed priorities, and schedulability analysis (RMS/EDF).	IEC 62304 Timing and implementation controls
ISR blocking or excessive interrupt latency	Nurse-call button becomes unresponsive during emergencies	ISR performs only minimal work, debounces the button, and wakes the responder task using Direct Task Notification instead of performing lengthy processing inside the interrupt.	IEC 62304 §5.5 – Safe software implementation practices
Race conditions between tasks	Incorrect patient state or inconsistent alarm decisions.	Queue transfers telemetry safely, Event Groups synchronize processing stages, and Direct Task Notifications provide one-to-one signaling, preventing unsafe shared-state communication.	IEC 62304 §5.5 – Controlled inter-process communication
False alarm generation	Alarm fatigue may reduce operator confidence.	Alarm is generated only after telemetry is processed, and rhythm thresholds indicate arrhythmia or hypoxemia. Coordinator ensures the complete processing pipeline finishes before responder activation.	IEC 62304 §5.7 – Software system testing and verification
Loss of system observability	Software failures become difficult to diagnose during operation.	Core 0 continuously reports queue depth, event bits, heartbeats, WCET, dropped samples, and alert counters without interfering with real-time execution on Core 1.	IEC 62304 §5.8 – Verification and maintenance support

Task execution exceeding deadline	Real-time pipeline could become nondeterministic and delay alarms.	Maximum WCET is measured for every task, utilization is calculated, and the task set is verified as schedulable under both RMS and EDF before deployment.	IEC 62304 §5.5 – Real-time implementation verification
Failure of monitoring interface	Loss of diagnostics even if the telemetry pipeline continues running.	Monitoring executes independently on Core 0 while the medical pipeline remains isolated on Core 1, preventing monitoring failures from affecting patient data processing.	EC 62304 §5.3 – Software architectural design and separation of concerns

Graceful Degradation:

If the telemetry processor can't keep up with the incoming data, then the queue will buffer up to 10 samples before additional samples become dropped. The system will detect if this happens through the queue depth as well as the dropped-sample counter while all the other real-time tasks are still running. This allows all the other telemetry information, alarms, and the nurse-call functionality to remain intact instead of causing a system wide failure.

Build & Run:

For this project, the tool chain is the ESP-IDF with FreeRTOS, and the board is ESP32-S3 which I ran via the WOKWI simulation. In order to run this simulation, you can run it via WOKWI or clone all the information that will be dropped in this repository to build it using ESP-IDF. When you run this simulation, you should see the serial monitor view with the live telemetry, task heartbeats, WCET measurements, and all other appropriate readings. Pressing the GPIO button

simulates a nurse-call interrupt, and then you can observe that notification updated on the serial monitor.

Tailored For:

This particular application is mainly for an embedded/real-time system. This project shows all of the skills we've acquired through the duration of this class. It focuses on core skills in embedded and firmware engineering including FreeRTOS task scheduling, dual-core programming, inter-process communication, interrupt handling, WCET analysis, watchdog monitoring, and the most important a deterministic real-time system design. All of these techniques are used to help us further understand the importance of a real-world example, in this case, it's the medical world.

What I copied from APP 1/2:

```
#if USE_WEBSERVER
/* =====
 * WEB MONITOR (Core 0) [USE_WEBSERVER = 1]
 * ===== */
static esp_err_t handle_root(httpd_req_t *req)
{
    UBaseType_t depth = uxQueueMessagesWaiting(data_q);
    EventBits_t bits = xEventGroupGetBits(evt_group);

    char buf[2048];
    int n = snprintf(buf, sizeof(buf),
        "<!DOCTYPE html>"
        "<html lang=\"en\"><head>"
        "<meta charset=\"utf-8\"><meta http-equiv=\"refresh\" content=\"1\">"
        "<title>ICU ECG Monitor · Dual-Core IPC Pipeline</title>"
        "<style>"
        "    body { font-family: -apple-system, sans-serif; background: #FAFAF5; "
        "        color: #1A1A1A; padding: 1.5rem; }"
```

```

" h1 { color: #6B4F09; border-bottom: 3px solid #FFC904; "
"     display: inline-block; padding-bottom: 4px; }"
" table { border-collapse: collapse; margin: 1rem 0; width: 100%; max-
width: 800px; }"
" th { background: #1A1A1A; color: #FFC904; padding: 8px 14px; "
"     text-align: left; font-size: 12px; text-transform: uppercase; }"
" td { padding: 8px 14px; border-bottom: 1px solid #ddd; }"
" td.num { font-variant-numeric: tabular-nums; font-weight: 700; color:
#6B4F09; }"
" .card { background: white; padding: 15px; margin-bottom: 15px; border-
radius: 6px; "
"         box-shadow: 0 2px 5px rgba(0,0,0,0.08); max-width: 800px; }"
" .alert { color: #e74c3c; font-weight: bold; }"
" .normal { color: #2ecc71; font-weight: bold; }"
"</style></head>"
"<body>"
"<h1>ICU ECG Telemetry &mdash; Dual-Core IPC Pipeline</h1>"
"<div class=\"card\">"
"  <h3>Current Telemetry Payload (Bed %lu)</h3>"
"  <p>Status: <span class=\"%s\">%s</span></p>"
"  <p><b>Heart Rate:</b> %u BPM | <b>SpO2:</b> %u%% | <b>ECG Wave:</b> %d
mV</p>"
"  <p><b>IPC Queue Depth:</b> %u / %d | <b>Event Bits:</b> 0x%02X</p>"
"</div>"
"<table>"
"<thead><tr><th>Task</th><th>Role</th><th>Period / Trigger</th>"
"<th>Priority</th><th>Heartbeats</th><th>WCET
(&micro;s)</th></tr></thead>"
"<tbody>"
"<tr><td>Task A &mdash; Telemetry Sampler</td><td>Producer</td>"
"  <td>50 ms</td><td>8</td>"
"  <td class=\"num\">%lu</td><td class=\"num\">%llu</td></tr>"
"<tr><td>Task B &mdash; Arrhythmia Process</td><td>Consumer</td>"
"  <td>Queue Event</td><td>8</td>"
"  <td class=\"num\">%lu</td><td class=\"num\">%llu</td></tr>"
"<tr><td>Task C &mdash; Pipeline Coordinator</td><td>Rendezvous</td>"
"  <td>Event Group</td><td>9</td>"
"  <td class=\"num\">%lu</td><td class=\"num\">%llu</td></tr>"
"<tr><td>Task D &mdash; Alarm Dispatcher</td><td>Actuator</td>"
"  <td>Notification</td><td>12</td>"
"  <td class=\"num\">%lu</td><td class=\"num\">%llu</td></tr>"
"</tbody></table>"
"<div class=\"card\">"

```

```

"    <p><b>Auto Arrhythmia Alerts:</b> %lu | <b>Manual Nurse Calls:</b> %lu
| <b>Dropped Samples:</b> %lu</p>"
    "</div>"
    "<p style='font-size: 0.85rem; color: #666;'\>Auto-refreshing every 1 s
&mdash; heartbeats must grow monotonically.</p>"
    "</body></html>",
    (unsigned long)last_sample.patient_id,
    (last_verdict == RHYTHM_ARRHYTHMIA) ? "alert" : "normal",
    (last_verdict == RHYTHM_ARRHYTHMIA) ? "ARRHYTHMIA DETECTED" : "RHYTHM
NORMAL",
    last_sample.heart_rate_bpm, last_sample.spo2_percent, last_sample.ecg_mv,
    (unsigned)depth, QUEUE_DEPTH, (unsigned)bits,
    (unsigned long)hb_a, (unsigned long long)wcet_a_max_us,
    (unsigned long)hb_b, (unsigned long long)wcet_b_max_us,
    (unsigned long)hb_c, (unsigned long long)wcet_c_max_us,
    (unsigned long)hb_d, (unsigned long long)wcet_d_max_us,
    (unsigned long)arrhythmia_alerts, (unsigned long)manual_nurse_calls,
(unsigned long)dropped_samples);

    httpd_resp_set_type(req, "text/html");
    httpd_resp_send(req, buf, n);
    return ESP_OK;
}

static httpd_handle_t start_webserver(void)
{
    httpd_config_t cfg = HTTPD_DEFAULT_CONFIG();
    cfg.server_port    = HTTP_PORT;
    cfg.core_id        = 0;
    cfg.task_priority  = 5;
    cfg.stack_size     = 8192;
    httpd_handle_t s    = NULL;
    if (httpd_start(&s, &cfg) == ESP_OK) {
        httpd_uri_t root = { .uri="/", .method=HTTP_GET,
                             .handler=handle_root, .user_ctx=NULL };
        httpd_register_uri_handler(s, &root);
        ESP_LOGI(TAG, "[WEBMON] HTTP server started on port %d", HTTP_PORT);
    }
    return s;
}

static void wifi_event_handler(void *arg, esp_event_base_t base,
                              int32_t id, void *data)

```

```

{
    if (base == WIFI_EVENT && id == WIFI_EVENT_STA_START)
        esp_wifi_connect();
    else if (base == WIFI_EVENT && id == WIFI_EVENT_STA_DISCONNECTED)
        esp_wifi_connect();
    else if (base == IP_EVENT && id == IP_EVENT_STA_GOT_IP) {
        ip_event_got_ip_t *e = (ip_event_got_ip_t *)data;
        ESP_LOGI(TAG, "[WEBMON] IP Address assigned: " IPSTR, IP2STR(&e-
>ip_info.ip));
        start_webserver();
    }
}

static void wifi_init_sta(void)
{
    ESP_ERROR_CHECK(nvs_flash_init());
    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());
    esp_netif_create_default_wifi_sta();
    wifi_init_config_t init = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&init));
    esp_event_handler_register(WIFI_EVENT, ESP_EVENT_ANY_ID,
wifi_event_handler, NULL);
    esp_event_handler_register(IP_EVENT, IP_EVENT_STA_GOT_IP,
wifi_event_handler, NULL);
    wifi_config_t cfg = { .sta = { .ssid = WIFI_SSID, .password = WIFI_PASS,
        .threshold.authmode = WIFI_AUTH_OPEN } };
    ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
    ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &cfg));
    ESP_ERROR_CHECK(esp_wifi_start());
}

static void webmonitor_task(void *arg)
{
    wifi_init_sta();
    for (;;) {
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

#else /* USE_WEBSERVER == 0 */

/* =====

```

```

*   TERMINAL MONITOR (Core 0) [USE_WEBSERVER = 0]
*   ===== */
static void task_monitor(void *arg)
{
    TickType_t last = xTaskGetTickCount();
    const TickType_t period = pdMS_TO_TICKS(1000);

    for (;;) {
        UBaseType_t depth = uxQueueMessagesWaiting(data_q);
        EventBits_t bits = xEventGroupGetBits(evt_group);

        printf("\n=== ICU ECG Telemetry Monitor · Dual-Core IPC Pipeline ===\n");
        printf("Telemetry: Status=%s | HR=%u BPM | SpO2=%u%% | ECG=%d mV\n",
            (last_verdict == RHYTHM_ARRHYTHMIA) ? "ARRHYTHMIA" : "NORMAL",
            last_sample.heart_rate_bpm, last_sample.spo2_percent,
last_sample.ecg_mv);
        printf("IPC State: Queue Depth=%u/%d | EventBits=0x%02X\n",
(unsigned)depth, QUEUE_DEPTH, (unsigned)bits);
        printf("%-30s %-8s %-9s %-12s %-10s\n",
            "Task", "Period", "Priority", "Heartbeats", "WCET(us)");
        printf("%-30s %-8s %-9d %-12lu %-10llu\n",
            "A: Telemetry Sampler (Prod)", "50 ms", 8, (unsigned long)hb_a,
(unsigned long long)wcet_a_max_us);
        printf("%-30s %-8s %-9d %-12lu %-10llu\n",
            "B: Arrhythmia Process (Cons)", "Queue", 8, (unsigned long)hb_b,
(unsigned long long)wcet_b_max_us);
        printf("%-30s %-8s %-9d %-12lu %-10llu\n",
            "C: Pipeline Coordinator", "Rendezvous", 9, (unsigned
long)hb_c, (unsigned long long)wcet_c_max_us);
        printf("%-30s %-8s %-9d %-12lu %-10llu\n",
            "D: Alarm Dispatch Actuator", "Notif", 12, (unsigned long)hb_d,
(unsigned long long)wcet_d_max_us);
        printf("Stats: Auto Alerts=%lu | Nurse Calls=%lu | Drops=%lu\n",
            (unsigned long)arrhythmia_alerts, (unsigned
long>manual_nurse_calls, (unsigned long)dropped_samples);

        vTaskDelayUntil(&last, period);
    }
}
#endif /* USE_WEBSERVER */

```

What I copied from APP 3:

```
static volatile int64_t last_edge_us = 0;
static void IRAM_ATTR button_isr(void *arg)
{
    int64_t now = esp_timer_get_time();
    if (now - last_edge_us < 200000) return; /* 200 ms software debounce */
    last_edge_us = now;

    manual_nurse_calls++;

    if (responder_handle != NULL) {
        BaseType_t woken = pdFALSE;
        vTaskNotifyGiveFromISR(responder_handle, &woken);
        portYIELD_FROM_ISR(woken);
    }
}
```

```
/* 4. Hardware Trigger ISR (Nurse Call Button) */
gpio_config_t cfg = {
    .pin_bit_mask = 1ULL << BUTTON_GPIO,
    .mode          = GPIO_MODE_INPUT,
    .pull_up_en    = GPIO_PULLUP_ENABLE,
    .intr_type     = GPIO_INTR_NEGEDGE,
};
gpio_config(&cfg);
gpio_install_isr_service(0);
gpio_isr_handler_add(BUTTON_GPIO, button_isr, NULL);
```

```
static void responder_task(void *arg)
{
    for (;;) {
        uint32_t n = ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
        if (n == 0) continue;

        uint64_t t_start = esp_timer_get_time();

        if (last_verdict == RHYTHM_ARRHYTHMIA) {
            ESP_LOGW(TAG, "[ACTUATOR] CRITICAL VITAL ALARM! Bed: %lu | HR: %u BPM
| SpO2: %u%% | Triggers: %lu",
```

```

        (unsigned long)last_sample.patient_id,
        last_sample.heart_rate_bpm,
        last_sample.spo2_percent,
        (unsigned long)n);
    } else {
        ESP_LOGI(TAG, "[ACTUATOR] Routine cycle check normal. Bed: %lu |
HR: %u BPM | SpO2: %u%%",
        (unsigned long)last_sample.patient_id,
        last_sample.heart_rate_bpm,
        last_sample.spo2_percent);
    }

    hb_d++;
    MEASURE_WCET(t_start, wcet_d_max_us);
}
}

```

```

/* Utility macro for WCET measurement */
#define MEASURE_WCET(start_time, max_var) do { \
    uint64_t elapsed = esp_timer_get_time() - start_time; \
    if (elapsed > max_var) { max_var = elapsed; } \
} while(0)

```

AI Disclosure:

- For this final assignment, I used my application 5 to complete this portfolio. Due to this, here is the link for all the AI that was used in creating Application 5:
 - <https://claude.ai/share/f894576d-e2f5-4796-92c5-66ab4d012b8b>

- I also used AI to help me build concurrency diagrams and to ensure that I correctly have all the information required by the rubric. :
 - <https://claude.ai/share/4937d5fe-8733-46cd-bfa3-cbf9383fc040>

- I also used ChatGPT to help me build my hazard analysis and standard mapping table based on all the information I already possessed and then convert that information to my .md file.
 - <https://chatgpt.com/share/6a6bef51-6c5c-83ea-8b15-94c92d89099d>
- I also used AI to help me build my index.html file so that everything gets displayed correctly.
 - <https://claude.ai/share/0c509195-ef29-4210-ab5a-2b9be6cc7f66>